THEORY

Training Program 1: Advanced NLP & Large Language Models (LLMs)

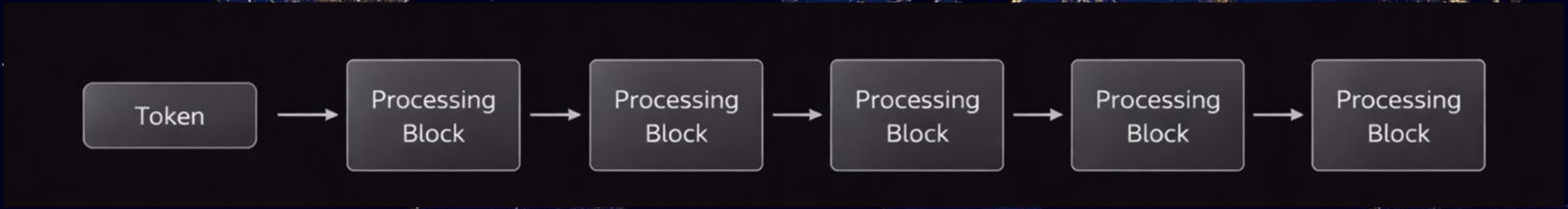
- M01: Text Preprocessing
- M02: Word and Sentence Embeddings
- M03: Transformer Architectures
- M04: Entity-Centric NLP
- M05:Applied NLP Tasks
- M06: Multimodal NLP
- M07: LLM Fundamentals
- M08: Prompt Engineering
- M09: Fine-Tuning & PEFT
- M10: RAG & Real-World Projects

THEORY

Advanced NLP & Large Language Models (LLMs)

M03 – Transformer Architectures

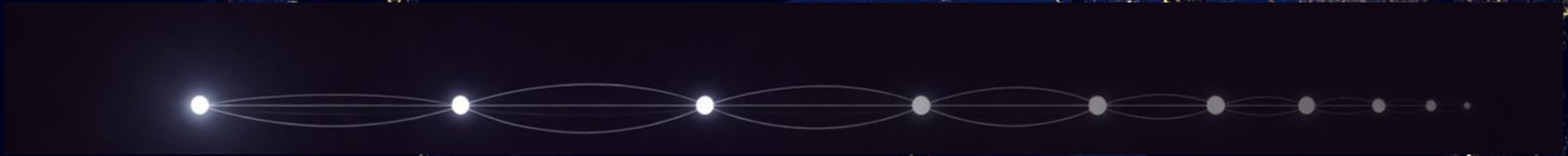
Chapter 3.1 – From RNNs to the Transformer Revolution



- RNNs process sequences token by token
- LSTMs and GRUs improved memory handling
- Dominant approach for sequence modeling
- Fundamentally sequential computation

M03 – Transformer Architectures

Chapter 3.1 – From RNNs to the Transformer Revolution

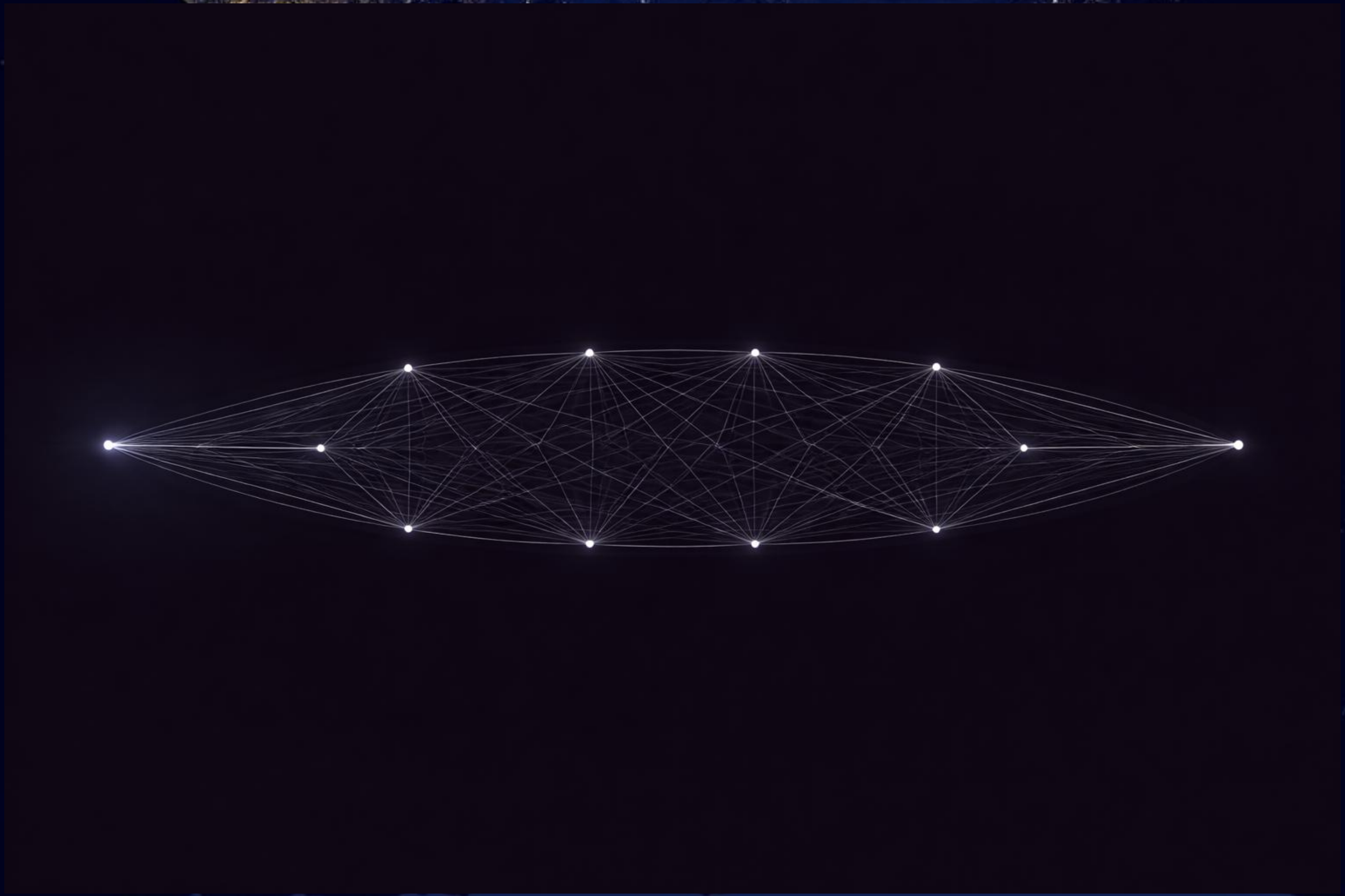


- No parallelization across sequence length
- Training is slow on long sequences
- Gradients decay over time steps
- Long-range dependencies are fragile

M03 – Transformer Architectures

Chapter 3.1 – From RNNs to the Transformer Revolution

- Tokens attend to each other directly
- No sequential bottleneck
- Full parallelization on GPUs
- Long-range relationships become trivial



THEORY

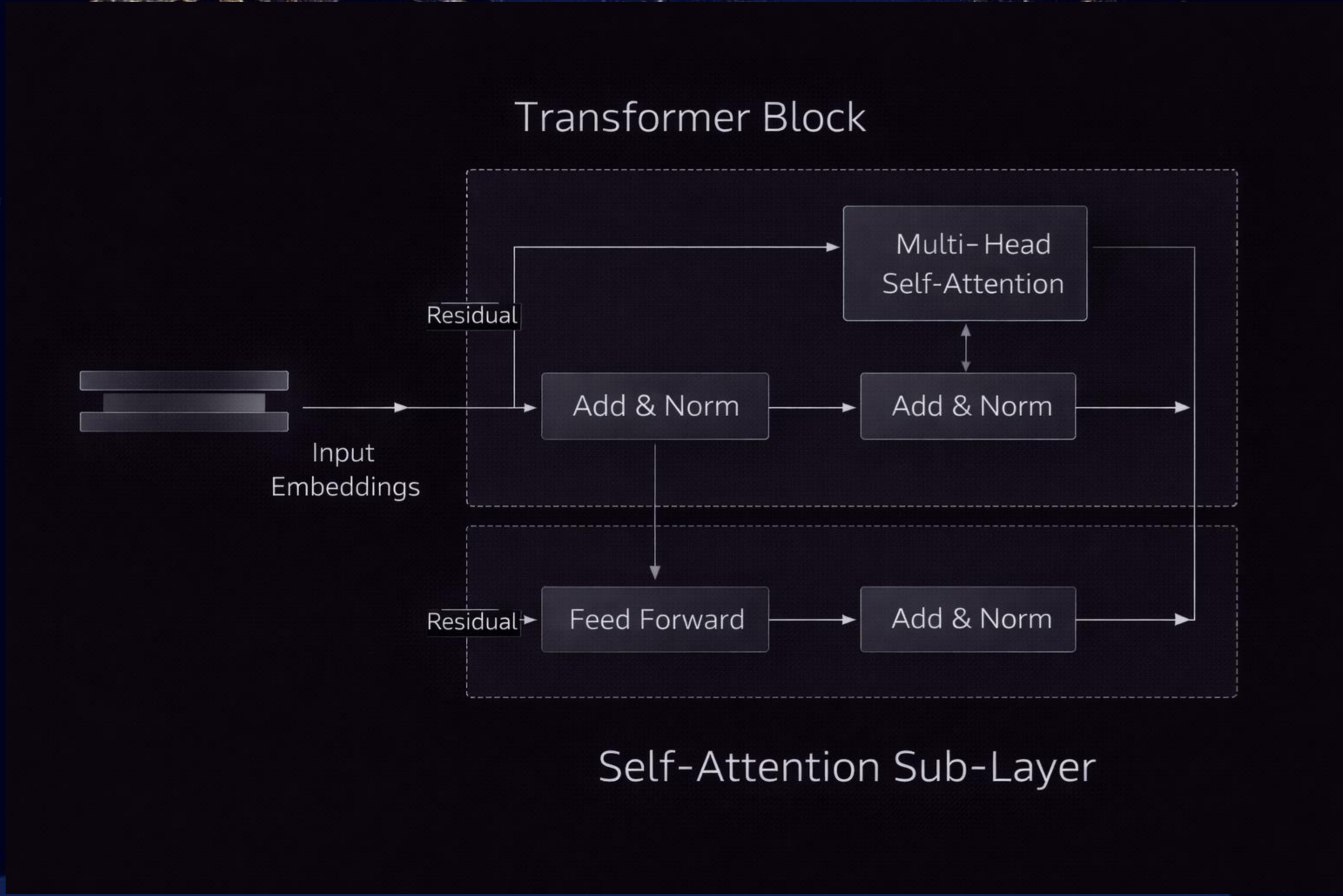
Advanced NLP & Large Language Models (LLMs)

M03 – Transformer Architectures

3.2 The Transformer: Attention Is All You Need

Self-Attention: Query, Key and Value

- Each token generates Query, Key and Value vectors
- Queries match against all Keys
- Values are weighted by attention scores
- Output is a context-aware representation

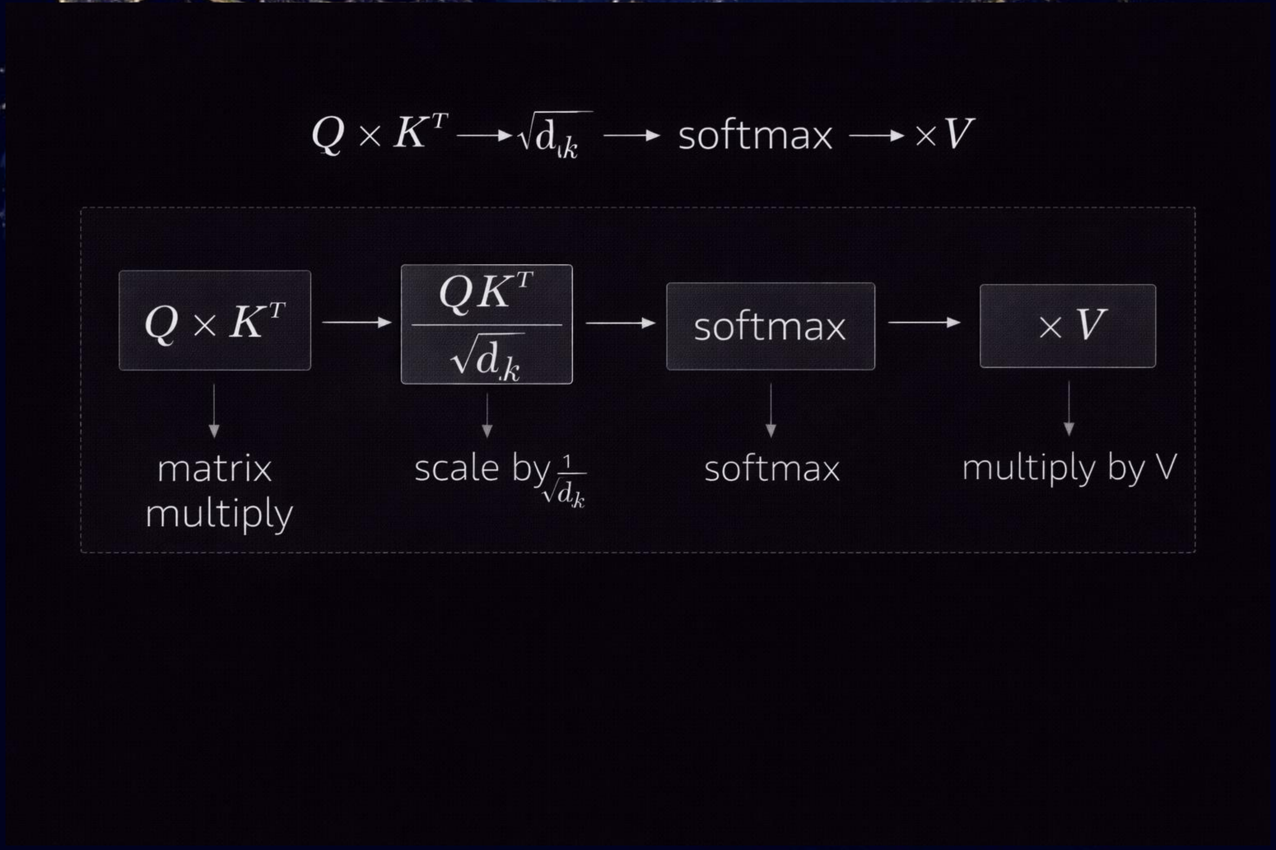


M03 – Transformer Architectures

3.2 The Transformer: Attention Is All You Need

Self-Attention: Query, Key and Value

- Dot product measures token compatibility
- Scores scaled by sqrt(d_k) for stability
- Softmax converts scores to probabilities
- Weighted sum produces attention output

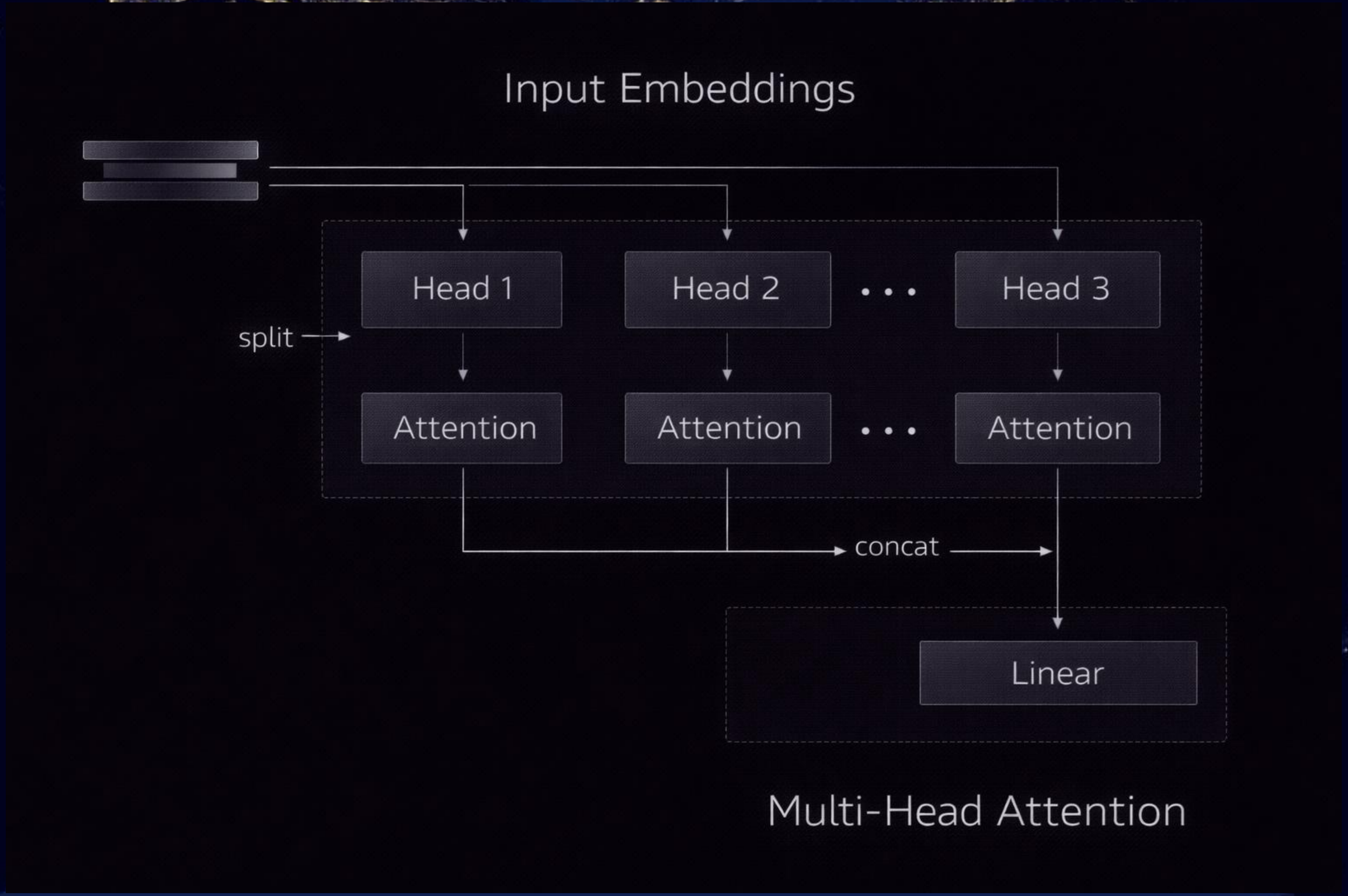


M03 – Transformer Architectures

3.2 The Transformer: Attention Is All You Need

Multi-Head Attention

- Attention is split into multiple heads
- Each head learns different relationships
- Heads run in parallel
- Outputs are concatenated and projected



THEORY

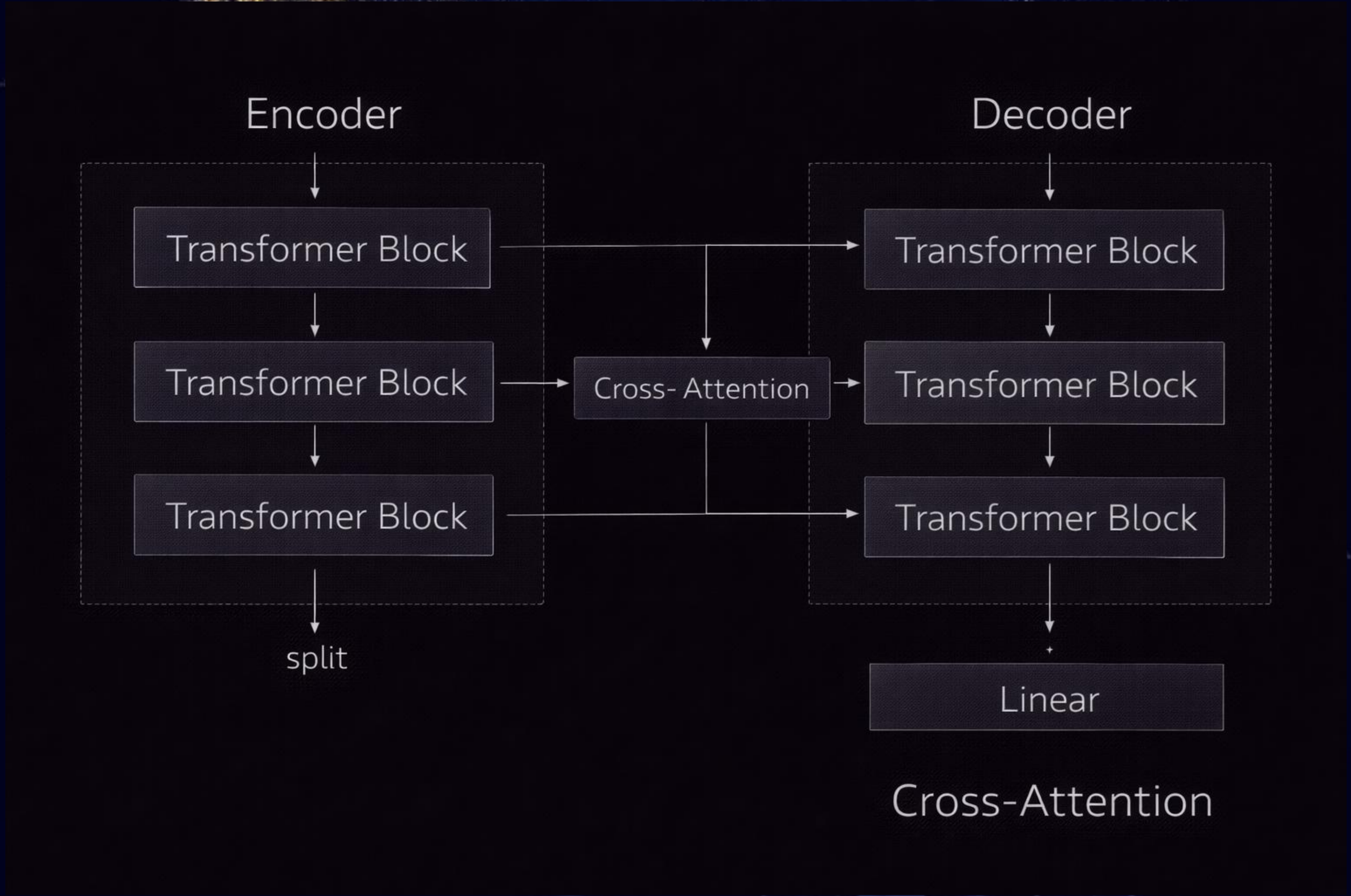
Advanced NLP & Large Language Models (LLMs)

M03 – Transformer Architectures

Chapter 3.3 – Encoder vs Decoder Architectures

Sequence-to-Sequence Transformer Overview

- Original Transformer uses Encoder–Decoder structure
- Encoder processes input sequence
- Decoder generates output sequence
- Cross-attention connects both stacks

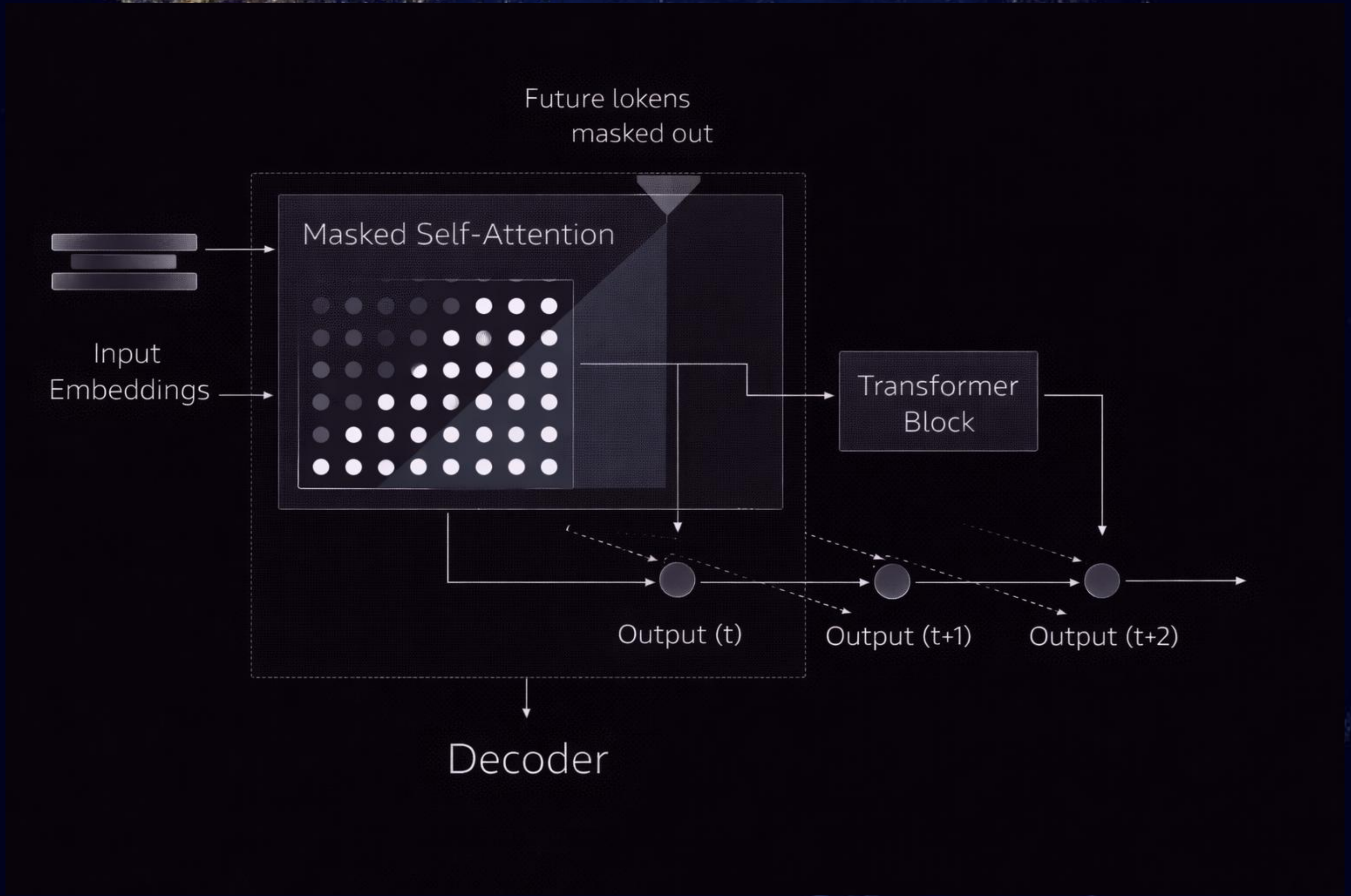


M03 – Transformer Architectures

Chapter 3.3 – Encoder vs Decoder Architectures

Encoder-Only Architecture (Understanding)

- Full bidirectional self-attention
- Tokens see left and right context
- Produces rich contextual representations
- Optimized for understanding tasks

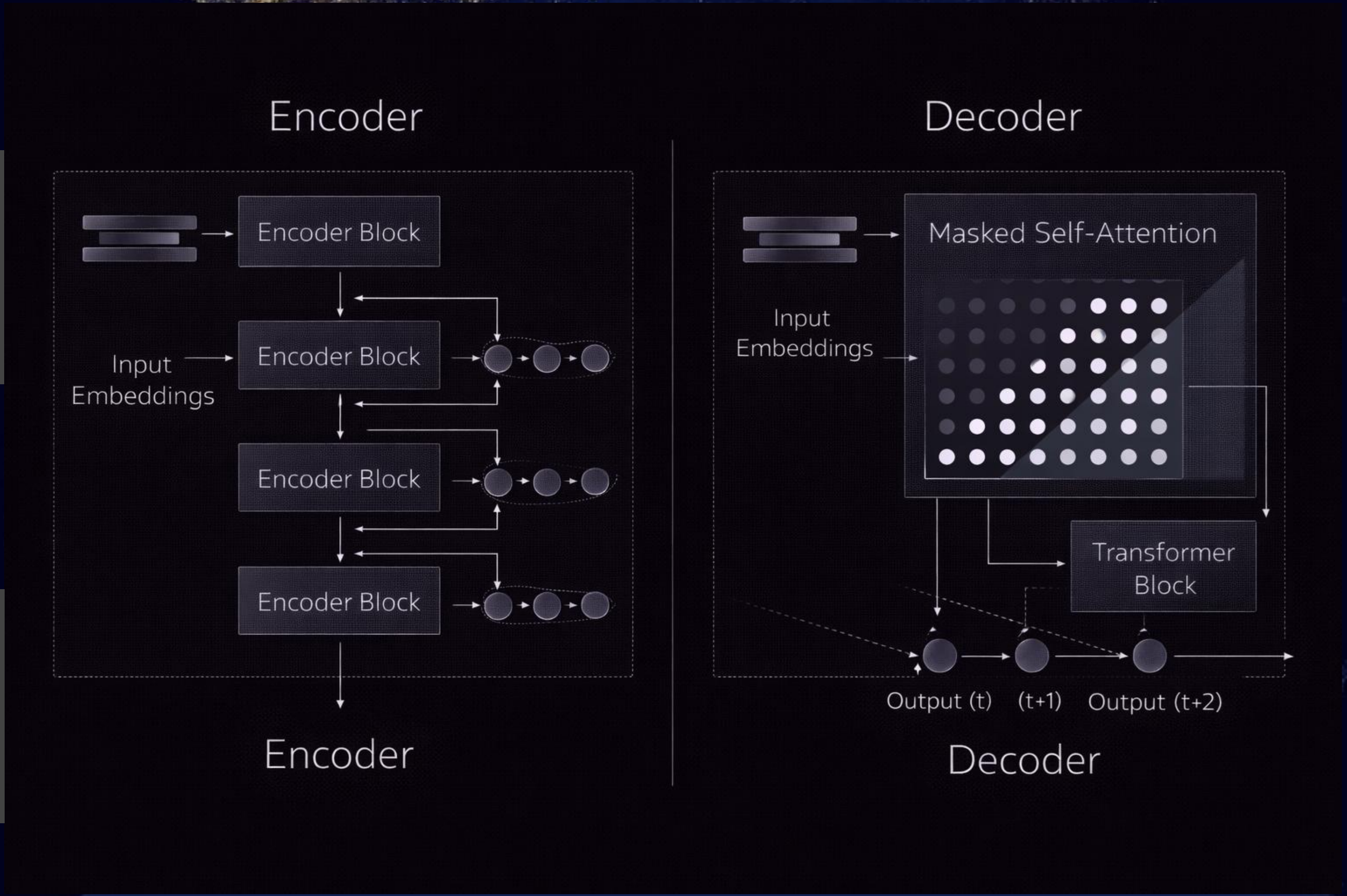


M03 – Transformer Architectures

Chapter 3.3 – Encoder vs Decoder Architectures

Decoder-Only Architecture (Generation)

- Causal (masked) self-attention
- Tokens only attend to past positions
- Auto-regressive generation
- Optimized for text generation



Architectural Comparison: Encoder vs Decoder

- Causal (masked) self-attention
- Tokens only attend to past positions
- Auto-regressive generation
- Optimized for text generation

CONFIANZA23
Inteligencia y Seguridad

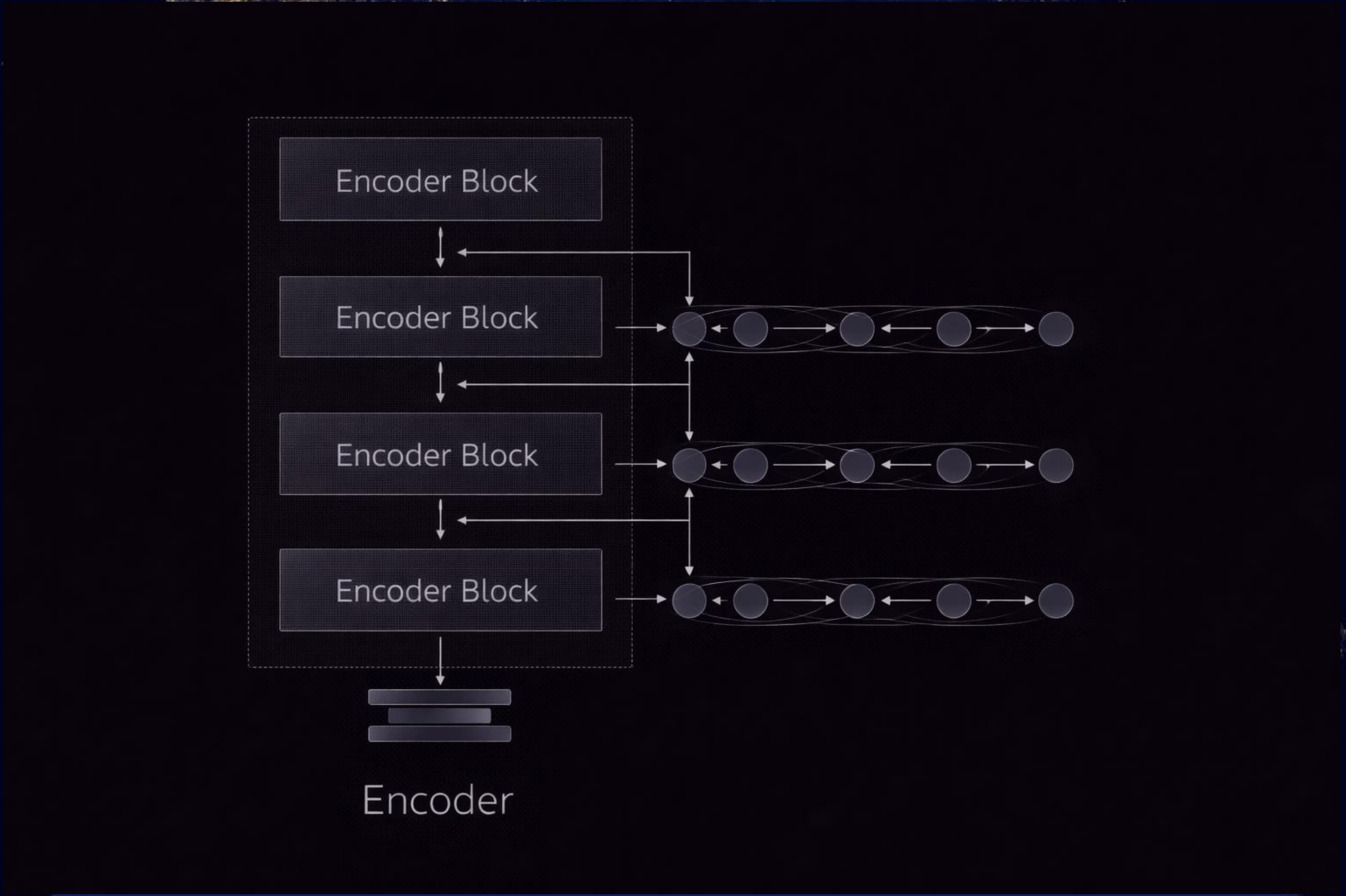
Advanced NLP & Large Language Models (LLMs)

M03 – Transformer Architectures

Chapter 3.4 – Encoder-Only Models

What Is an Encoder-Only Transformer?

- Uses only the encoder stack
- Full bidirectional self-attention
- Builds deep contextual representations
- Designed for understanding tasks



M03 – Transformer Architectures

Chapter 3.4 – Encoder-Only Models

BERT: Bidirectional Context Modeling

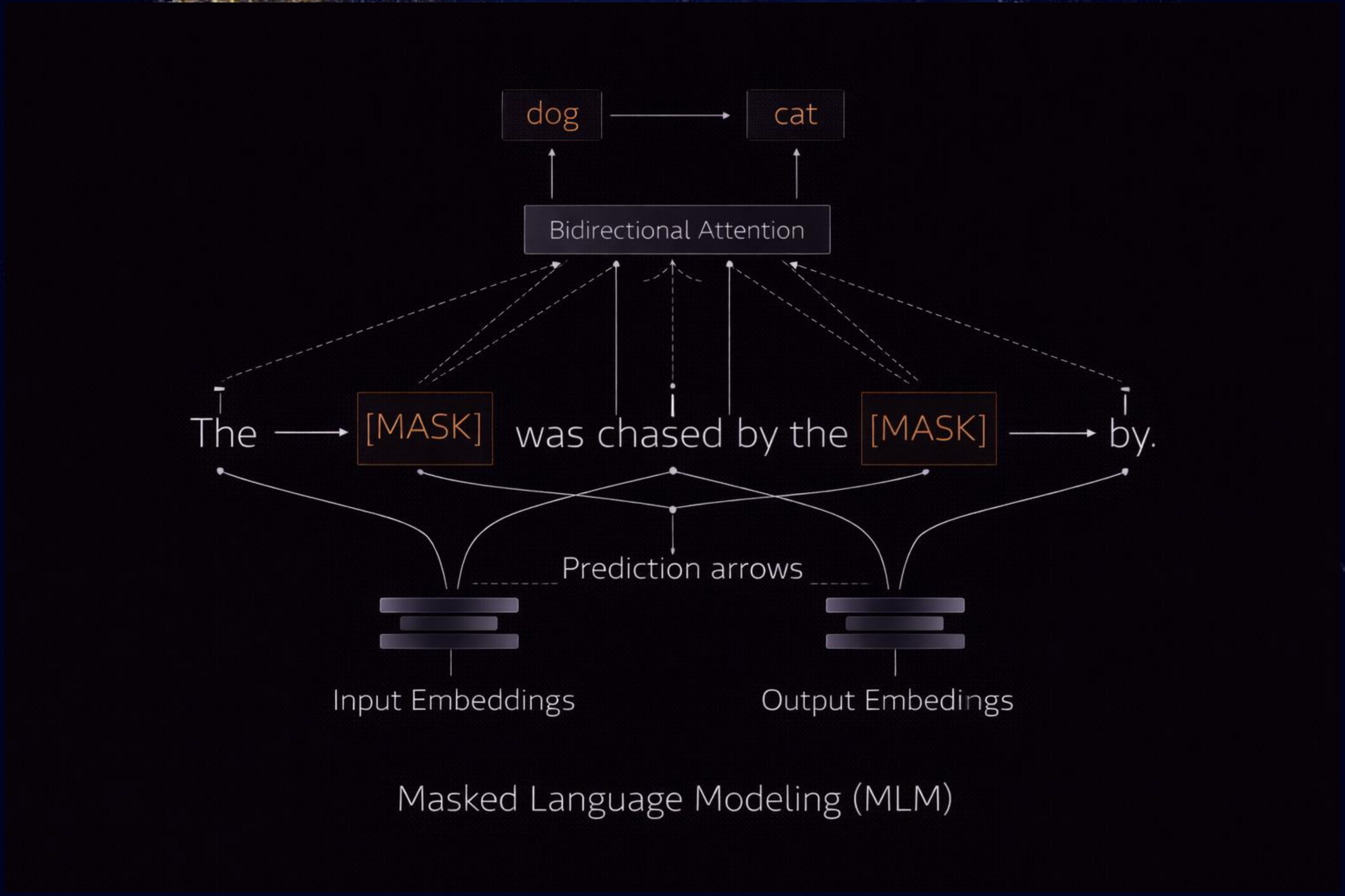
- Tokens attend left and right simultaneously
- Meaning depends on full sentence context
- Same word gets different vectors per context
- Solves polysemy effectively

M03 – Transformer Architectures

Chapter 3.4 – Encoder-Only Models

Typical Use Cases of Encoder-Only Models

- Text classification
- Sentiment analysis
- Named Entity Recognition
- Semantic similarity scoring

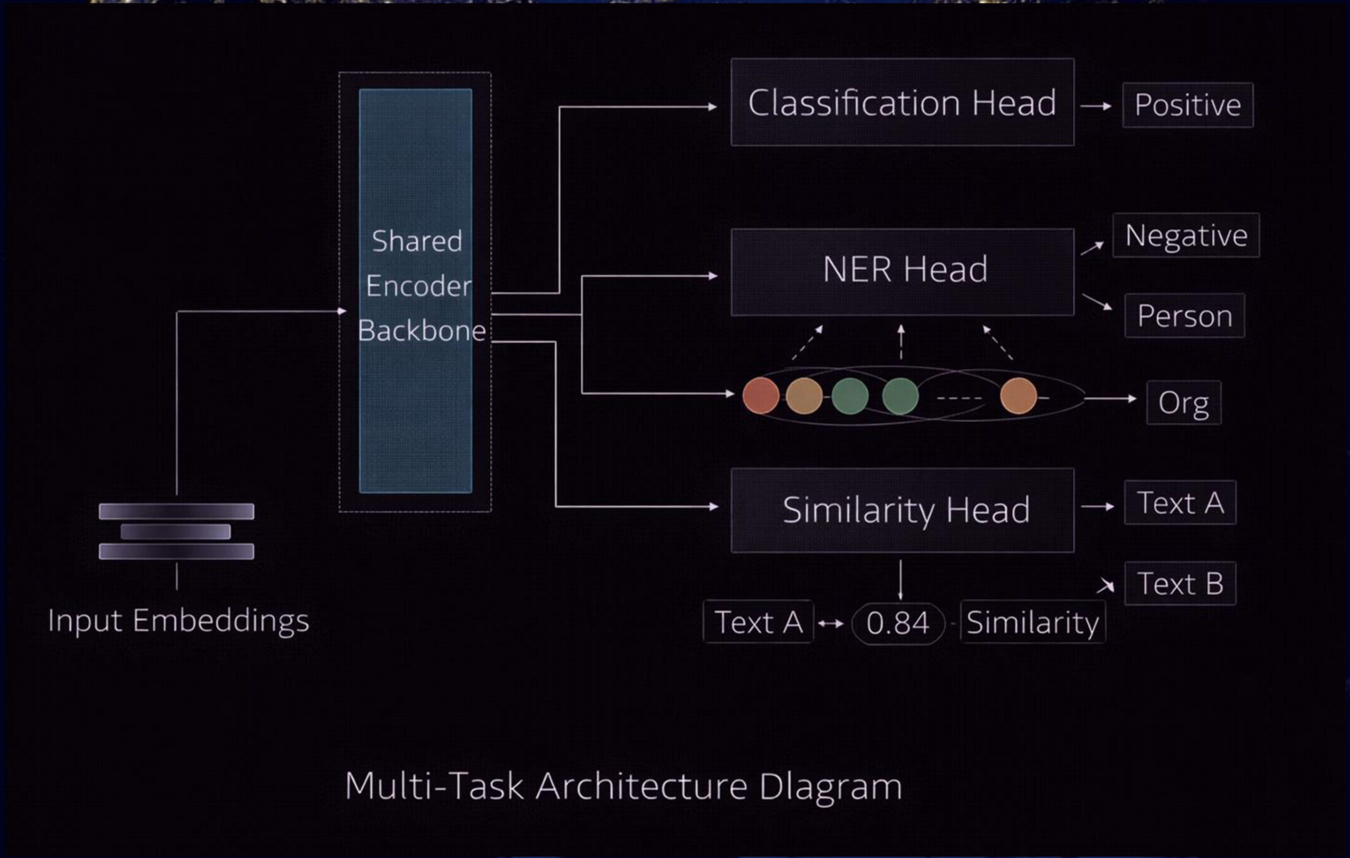


M03 – Transformer Architectures

Chapter 3.4 – Encoder-Only Models

Typical Use Cases of Encoder-Only Models

- Text classification
- Sentiment analysis
- Named Entity Recognition
- Semantic similarity scoring



THEORY

CONFIANZA23
Inteligencia y Seguridad

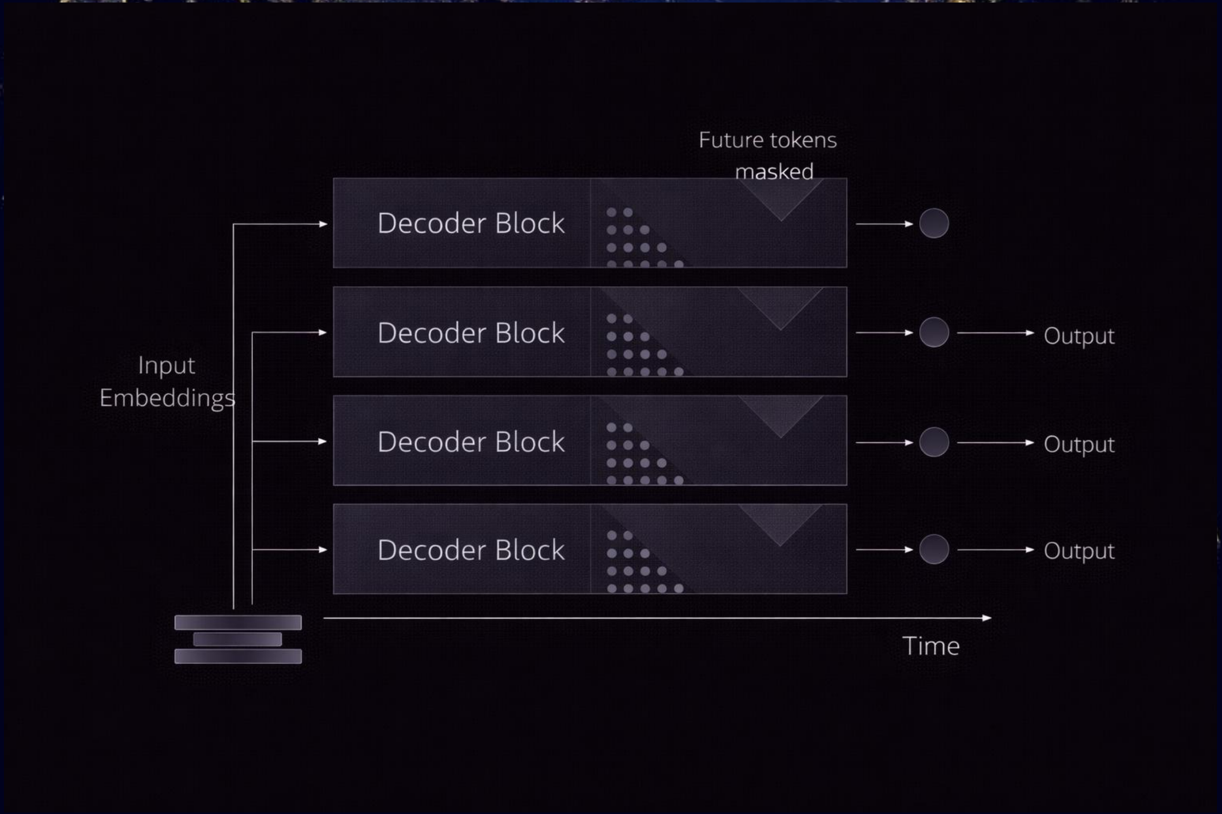
Advanced NLP & Large Language Models (LLMs)

M03 – Transformer Architectures

Chapter 3.5 – Decoder-Only Models

What Is a Decoder-Only Transformer?

- Uses only the decoder stack
- No bidirectional attention
- Generates text token by token
- Optimized for sequence generation

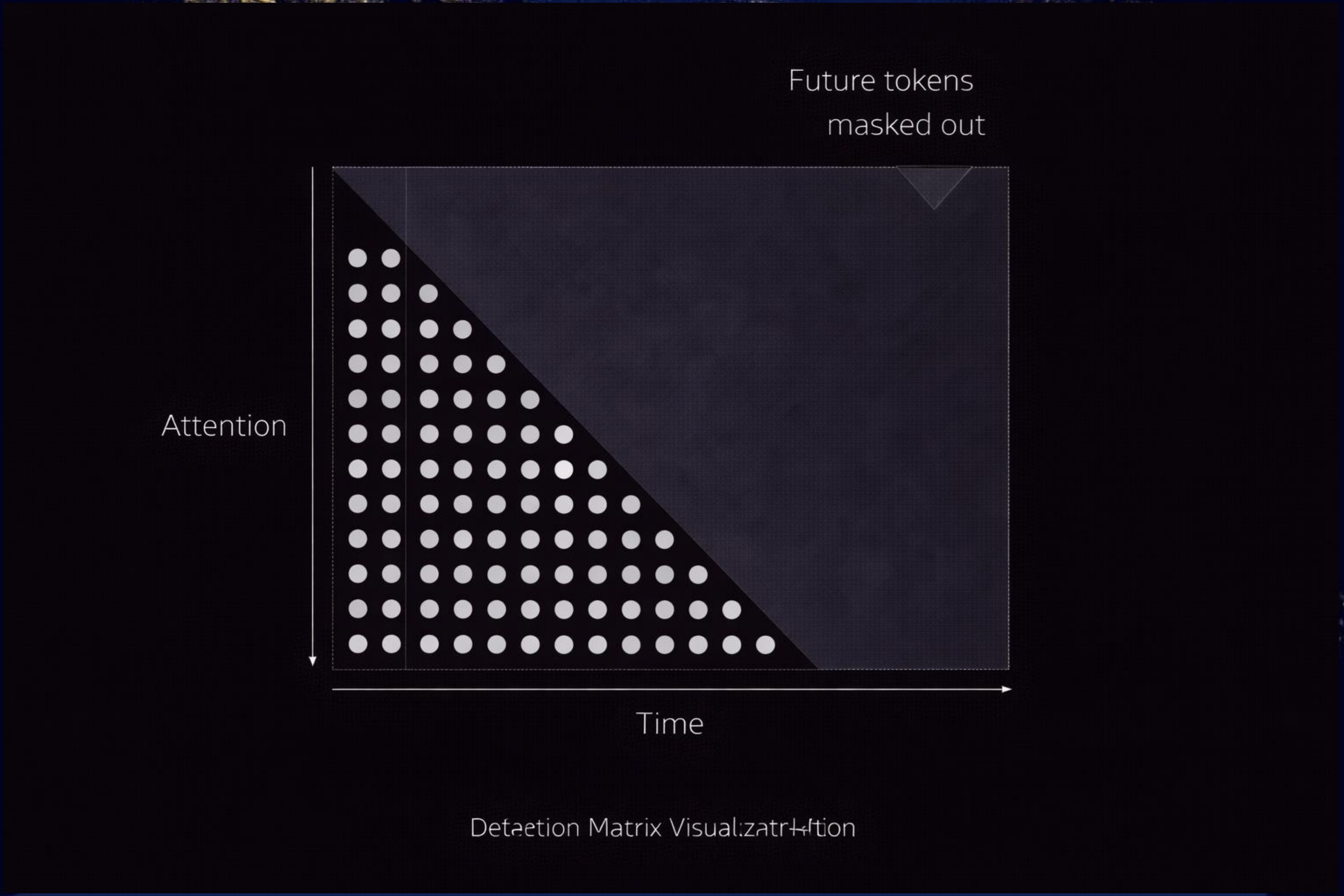


M03 – Transformer Architectures

Chapter 3.5 – Decoder-Only Models

Causal (Masked) Self-Attention

- Tokens can only attend to past tokens
- Future positions are masked
- Prevents information leakage
- Enables auto-regressive learning

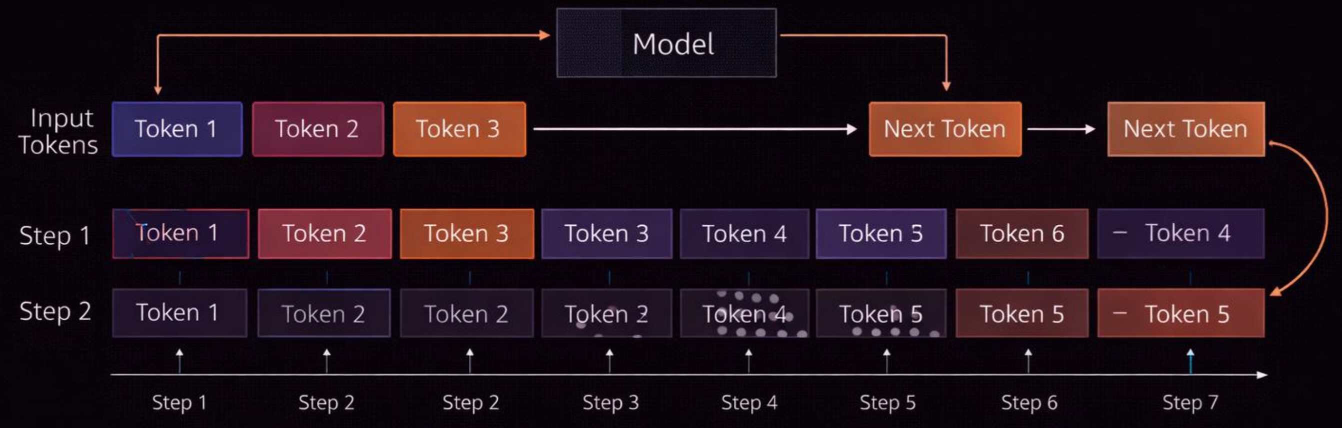


M03 – Transformer Architectures

Chapter 3.5 – Decoder-Only Models

Auto-Regressive Generation Loop

- Predict next token probability
- Append generated token to input
- Repeat until stop condition
- Text emerges sequentially



Auto-Regressive Generation Loop

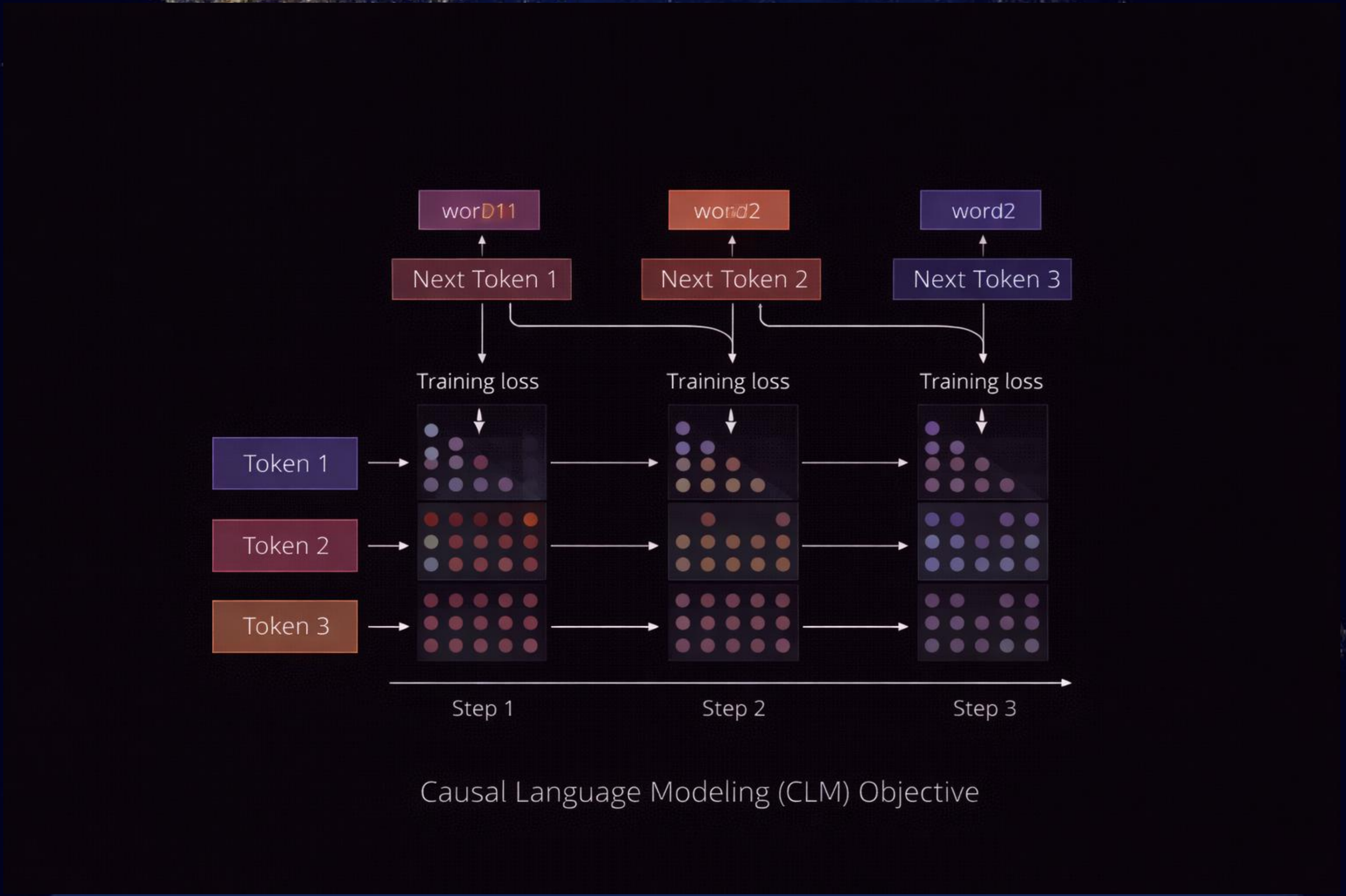
M03 – Transformer Architectures

Chapter 3.5 – Decoder-Only Models

GPT Training Objective

- Causal Language Modeling (CLM)
- Predict token t+1 given tokens ≤ t
- Trained on massive unlabeled text
- Simple objective, emergent intelligence

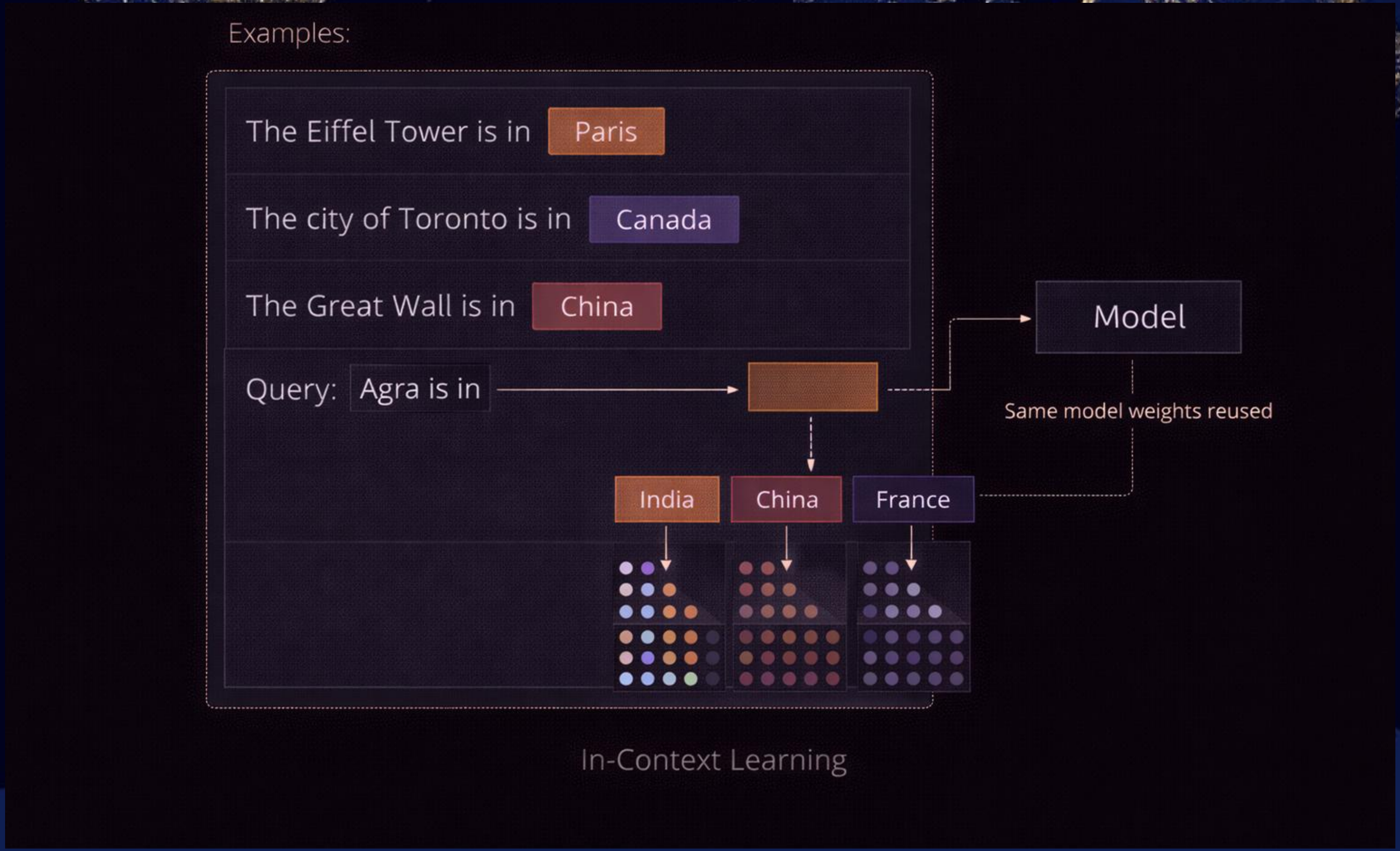
- Model learns from examples in prompt
- No weight updates required
- Task defined by context
- Emergent property of scale



M03 – Transformer Architectures

Chapter 3.5 – Decoder-Only Models

Typical Use Cases of Decoder-Only Models



THEORY

CONFIANZA23
Inteligencia y Seguridad

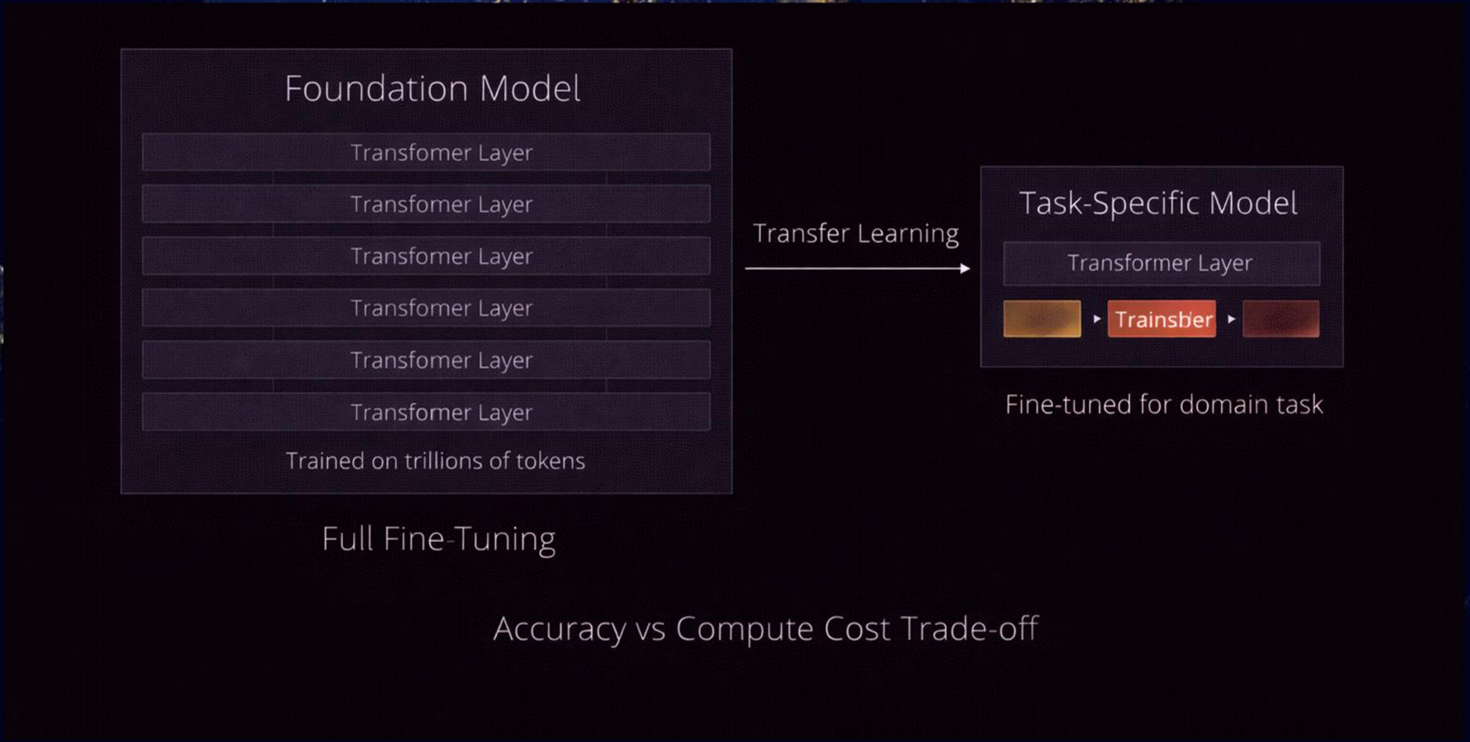
Advanced NLP & Large Language Models (LLMs)

M03 – Transformer Architectures

Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

What Is Transfer Learning?

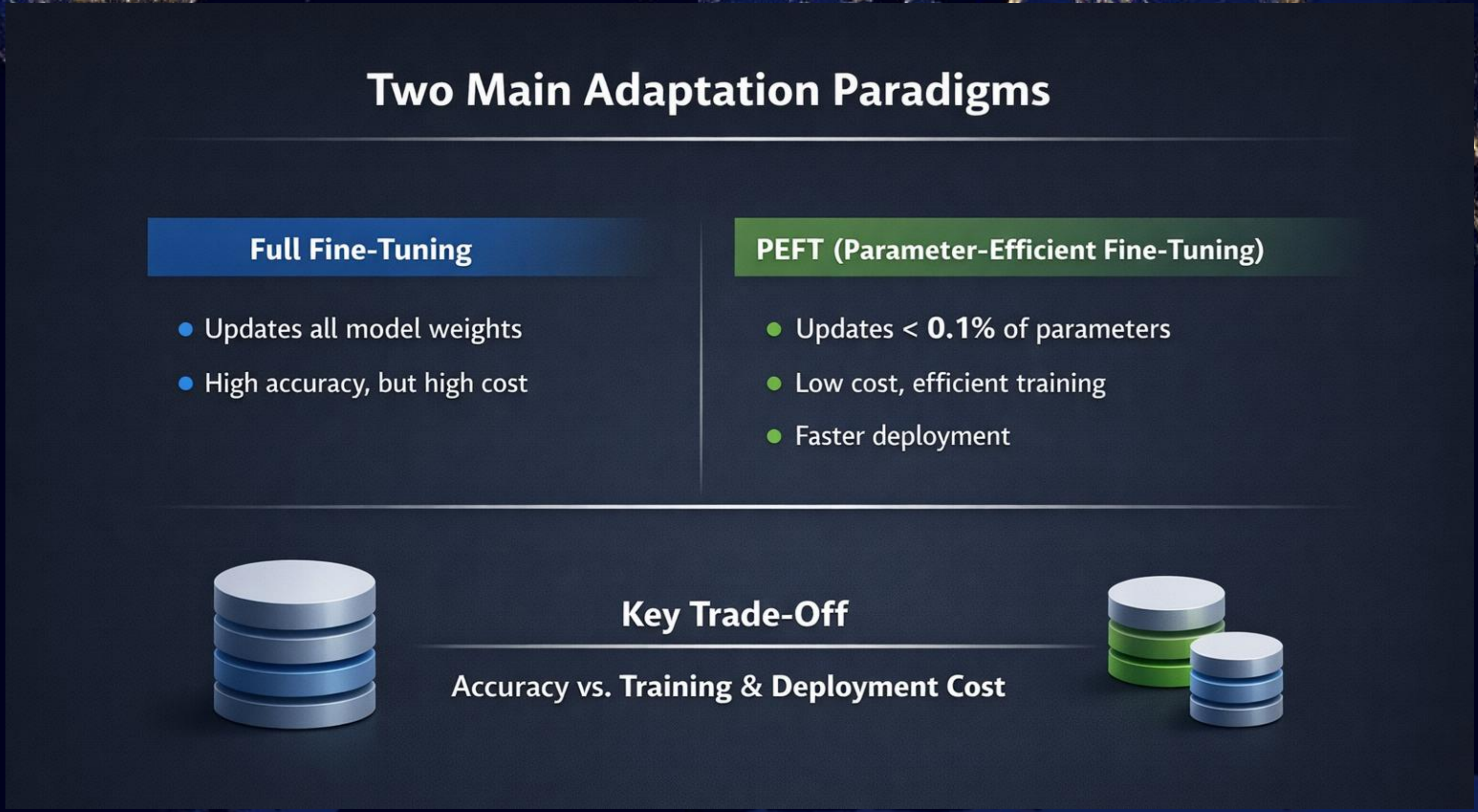
- Reuses a large pre-trained foundation model
- Avoids training from scratch
- Requires much less data and compute
- Enables specialization (law, medicine, security)



M03 – Transformer Architectures

Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

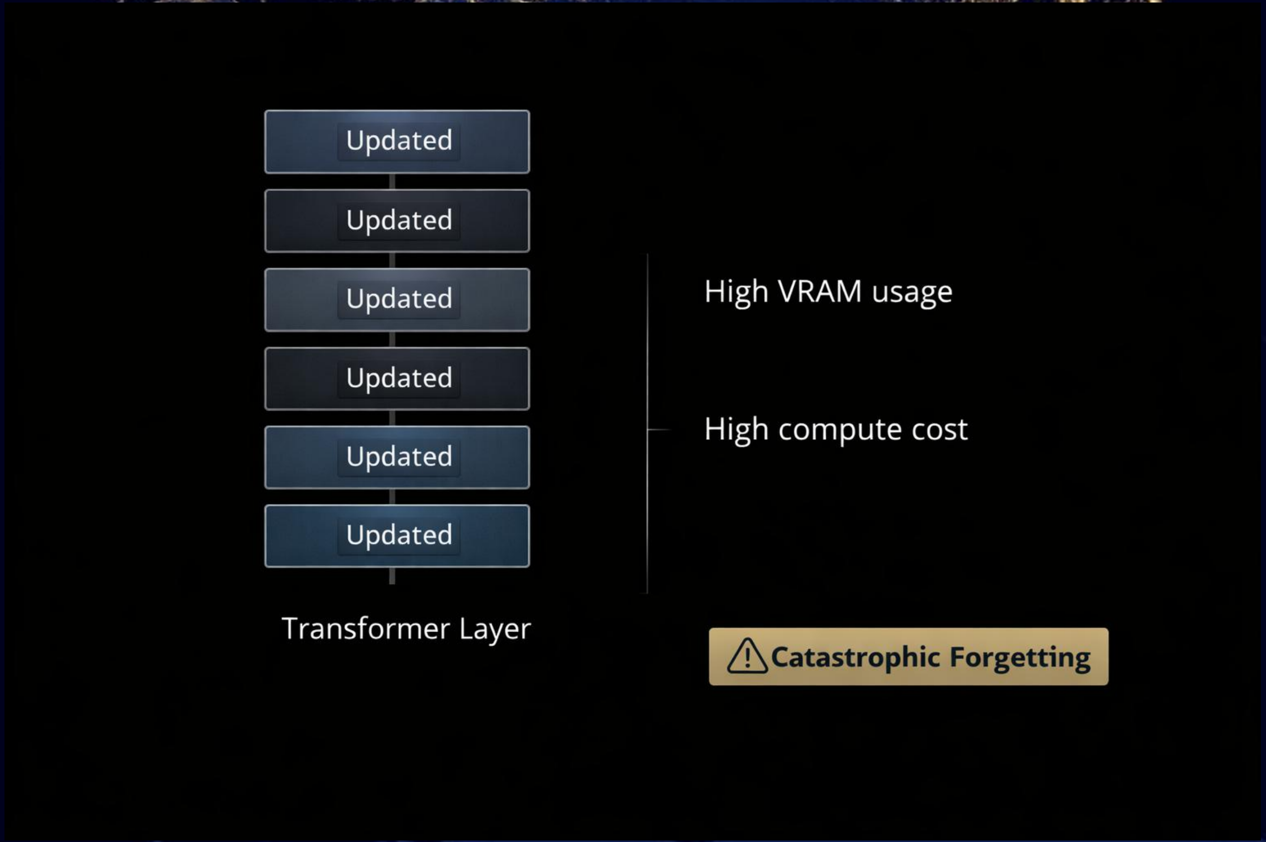
Full Fine-Tuning vs Parameter-Efficient Fine-Tuning



Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

Full Fine-Tuning: Characteristics

- Updates every parameter in the model
- Highest adaptation flexibility
- Very high GPU and VRAM usage
- Risk of catastrophic forgetting

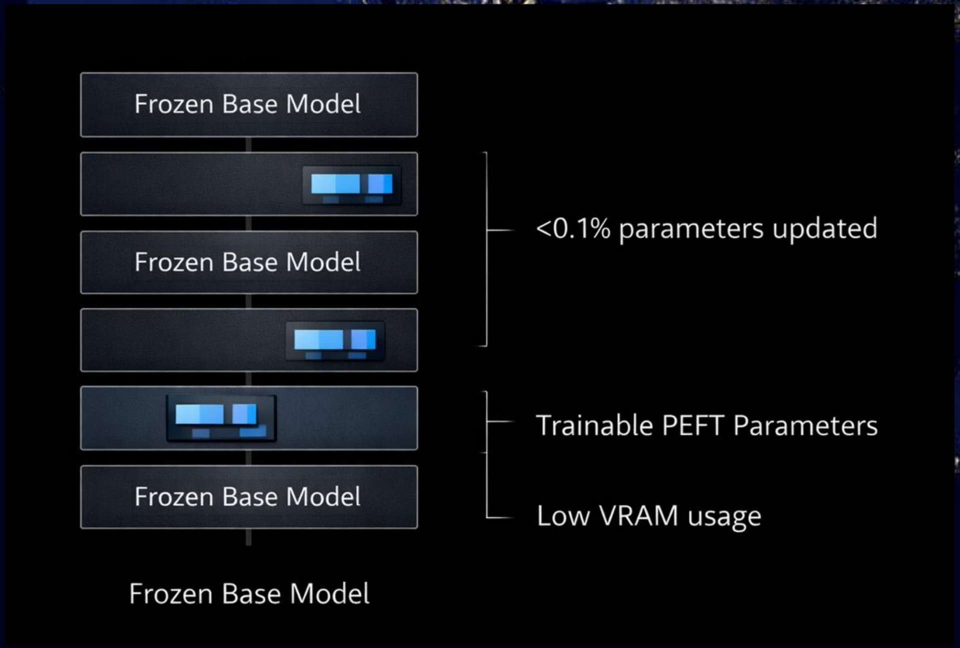


M03 – Transformer Architectures

Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

PEFT: Parameter-Efficient Fine-Tuning

- Base model weights are frozen
- Only small parameter blocks trained
- Preserves original model knowledge
- Much lower memory requirements

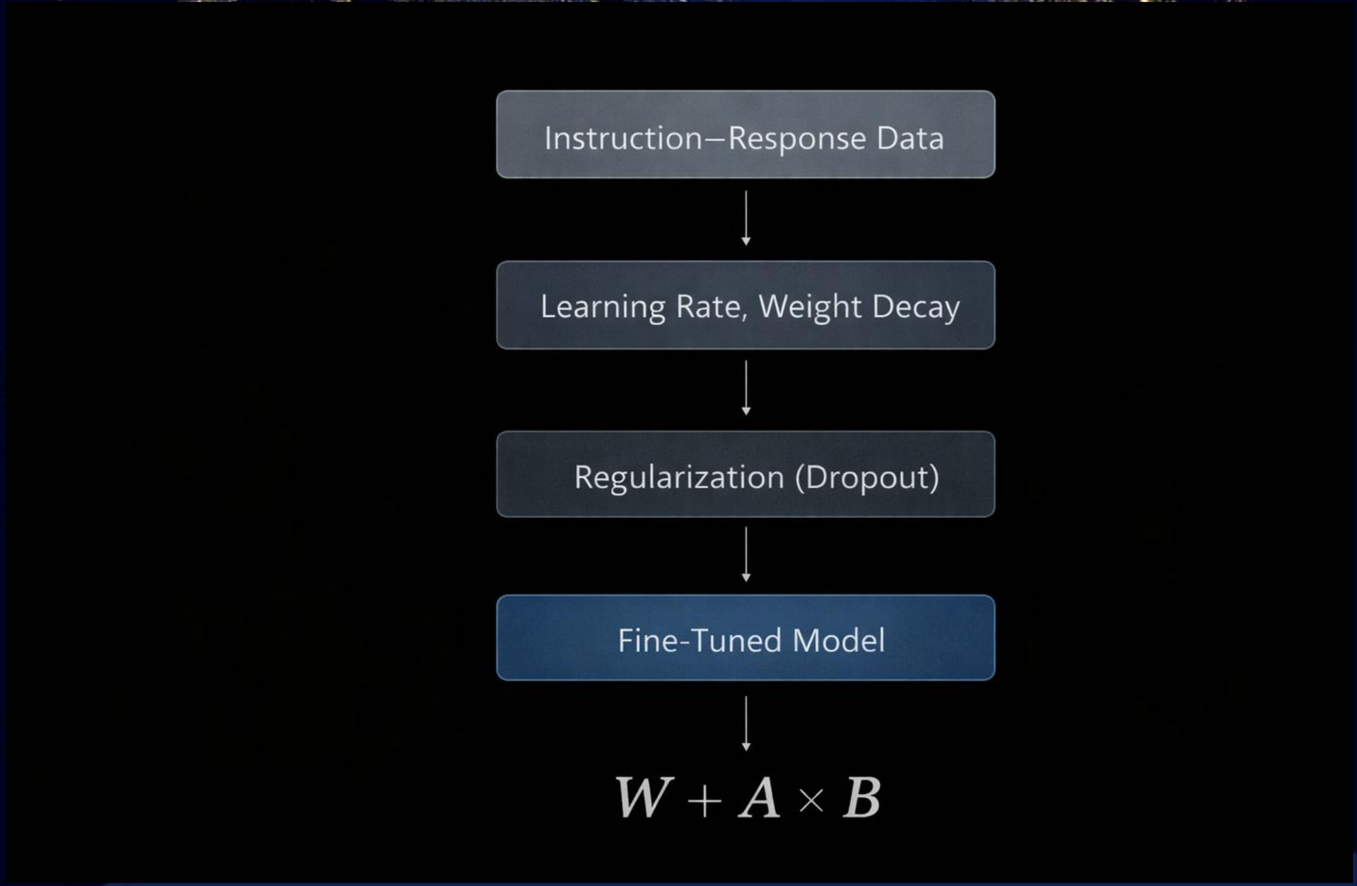


M03 – Transformer Architectures

Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

Choosing the Right Architecture

- Injects low-rank matrices into attention layers
- Only LoRA parameters are trained
- Base weights remain unchanged
- No inference latency penalty



THEORY

CONFIANZA23
Inteligencia y Seguridad

Advanced NLP & Large Language Models (LLMs)

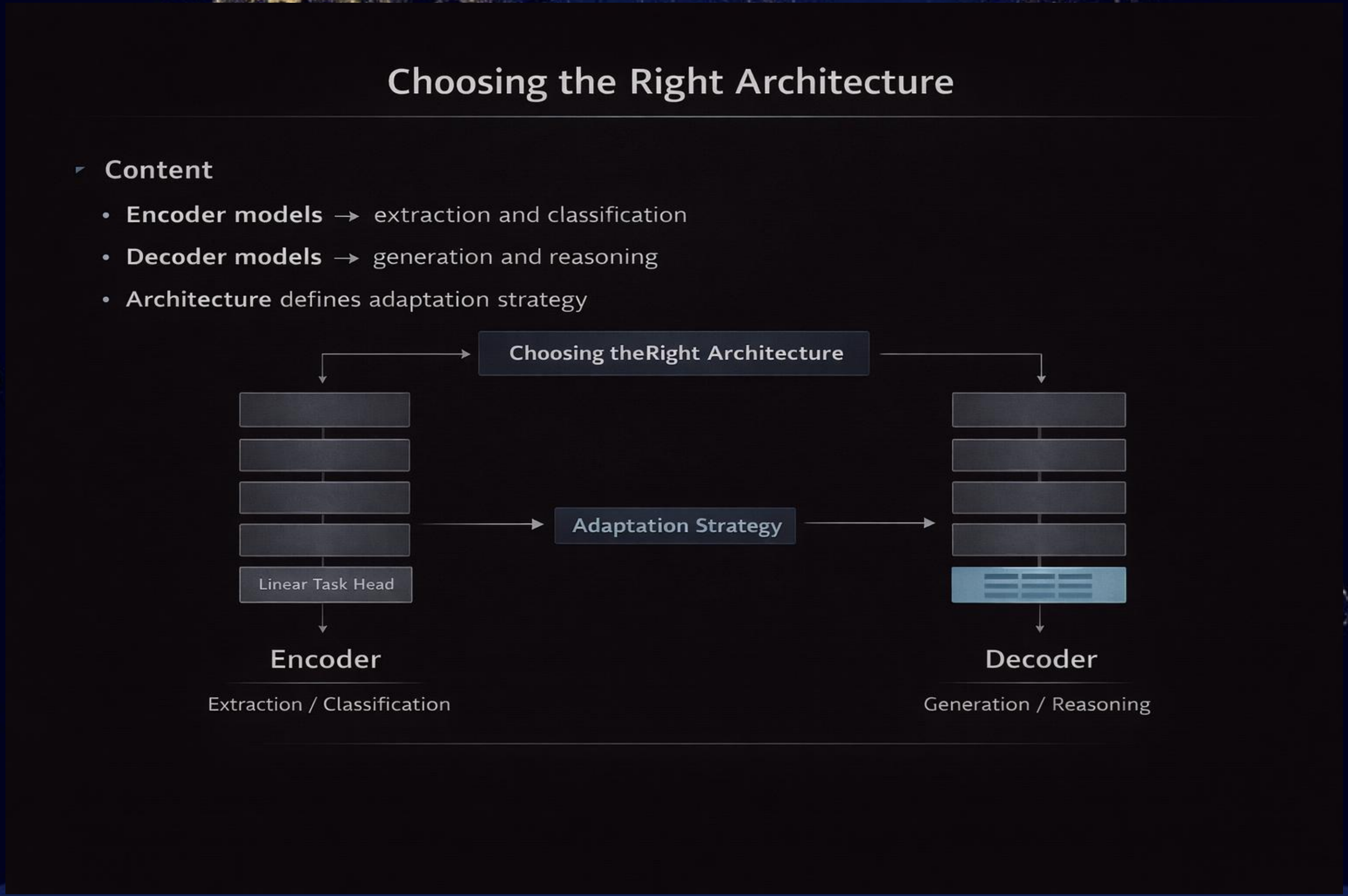
M03 – Transformer Architectures

Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

Choosing the Right Architecture

- RNNs introduced sequence modeling
- Transformers removed recurrence limits
- Attention enabled global context
- Scale unlocked emergent intelligence

- Attention is the core computation
- Encoders specialize in understanding
- Decoders specialize in generation
- Architecture defines capabilities



Chapter 3.6 – Fine-Tuning Strategies & PEFT (LoRA)

Choosing the Right Architecture

- Vaswani et al. (2017) – Attention Is All You Need
- Devlin et al. (2018) – BERT: Pre-training of Deep Bidirectional Transformers
- Brown et al. (2020) – Language Models are Few-Shot Learners
- Hu et al. (2021) – LoRA: Low-Rank Adaptation of Large Language Models
- Jay Alammar – The Illustrated Transformer & GPT Series
- Hugging Face – Transformers & PEFT Documentation
- Stanford CS224N – Transformer & Attention Lectures